

Architecture de flashage ESP32 (Raspberry Pi & QEMU)

1. Introduction

Ce document explique le processus de flashage d'un ESP32 depuis un Raspberry Pi (ou PC) et compare ce fonctionnement avec une exécution sous QEMU.

On distingue deux environnements :

-  **ESP32 réel (hardware)**
 -  **ESP32 simulé (QEMU)**
-

2. Architecture de flashage ESP32 réel

2.1 Vue globale

Raspberry Pi / PC



USB / UART



USB-UART Bridge (CH340 / CP2102)



ESP32 ROM Bootloader



SPI Flash (externe)

2.2 Rôle des composants

Boot ROM (interne ESP32)

- Gravée en usine
- Lance le mode boot
- Déclenche le mode download si GPIO0 = LOW

SPI Flash (mémoire externe)

Contient :

- bootloader.bin
 - partition-table.bin
 - application.bin
-

2.3 Séquence de flashage

Étape 1 — Mode flash

Pour entrer en mode téléchargement :

GPIO0 = LOW

RESET pulse

Étape 2 — Communication UART

Le Boot ROM active :

UART0 (Serial Bootloader)

Étape 3 — Envoi des binaires

via esptool.py :

bootloader.bin → 0x1000
partition-table → 0x8000
app.bin → 0x10000

2.4 Organisation mémoire flash

0x1000 → Bootloader
0x8000 → Partition Table
0x10000 → Application

2.5 Résultat final

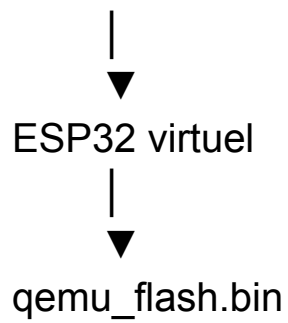
Après reset :

Boot ROM
↓
Bootloader
↓
Partition Table
↓
Application

3. Architecture flashage QEMU

3.1 Vue globale

Raspberry Pi / PC
↓
▼
QEMU Emulator



3.2 Différence principale

ESP32 réel	QEMU
SPI Flash physique	fichier <code>.bin</code>
UART matériel	UART virtuel
Boot ROM réelle	Boot ROM simulée

3.3 Flash dans QEMU

QEMU ne flashe pas réellement.

On crée une image unique :

qemu_flash.bin

Elle contient :

- bootloader
 - partition table
 - application
-

3.4 merge_bin

Utilisé pour regrouper les binaires :

esptool.py merge_bin

Objectif :

👉 produire une image flash complète

4. Démarrage QEMU

Commande typique :

```
qemu-system-xtensa \  
-machine esp32 \  
-drive file=qemu_flash.bin \  
-serial mon:stdio
```

Processus :

CPU Xtensa virtuel



Boot ROM simulée



Lecture qemu_flash.bin



Bootloader



Application

5. Simulation interne ESP32 dans QEMU

5.1 CPU Xtensa

QEMU interprète les instructions Xtensa :

```
add a4, a2, a3
```

Puis les traduit en instructions CPU hôte via TCG :

Xtensa → TCG → x86/ARM

5.2 Mémoire RAM

ESP32 address space



QEMU memory array

Exemple :

0x3FFAE000 → ram[]

5.3 Flash SPI

Dans QEMU :

qemu_flash.bin

Remplace la mémoire SPI réelle.

5.4 GPIO

GPIO write → QEMU handler → état interne

Exemple :

GPIO2 = 1 → gpio[2] = ON

5.5 UART

ESP32 UART → QEMU → stdout PC

Donc :

```
printf("Hello");
```

apparaît directement dans le terminal.

5.6 Interruptions & Timers

- Interruptions → événements logiciels QEMU
 - Timers → basés sur horloge hôte
-

5.7 Limitations QEMU

- ✗ WiFi
- ✗ Bluetooth
- ✗ ADC/DAC précis
- ✗ SPI complexe temps réel



6. Comparaison globale

ESP32 réel

Code



Build



binaires



UART (esptool)



SPI Flash



Exécution hardware

QEMU

Code



Build



binaires



merge_bin



qemu_flash.bin



Exécution CPU virtuel



7. Architecture ESP32 complète

7.1 Composants hardware

- CPU Xtensa LX6
 - ROM Boot
 - RAM
 - SPI Flash controller
 - UART / GPIO
 - Timers / Interrupts
-

7.2 Boot sequence réel

Power ON



Boot ROM



GPIO0 check



SPI Flash bootloader



Partition table



Application



8. Flashing ESP32 avec Raspberry Pi

Rôle du Raspberry Pi :

- contrôle RESET + GPIO0
- envoi UART
- exécute esptool

Raspberry Pi



UART + GPIO



ESP32 Boot ROM



SPI Flash écrite



9. ESP-IDF et compilation

9.1 ESP-IDF

SDK officiel Espressif :

- drivers
 - bootloader
 - outils flash
-

9.2 Compilation

idf.py build

Produit :

- bootloader.bin
 - partition-table.bin
 - app.bin
-



10. Bootloader et partition table

Bootloader

Petit programme qui :

- initialise le système
- charge l'application

Partition table

Définit la structure mémoire :

0x1000 bootloader

0x8000 partition table

0x10000 application

11. Watchdog (résumé propre)

Rôle

Le watchdog surveille le système.

si CPU bloque → RESET

Types

- Task Watchdog
 - Interrupt Watchdog
 - RTC Watchdog
-

Pourquoi le désactiver en dev ?

- éviter resets intempestifs
 - faciliter tests bare-metal
-

Bonne pratique

-  désactiver en dev avancé
 -  activer en production
-



Conclusion

La différence fondamentale entre ESP32 réel et QEMU est :

- ESP32 réel → mémoire SPI physique
- QEMU → fichier `qemu_flash.bin`

Tout le reste (boot ROM, adresses, bootloader, CPU) est reproduit pour simuler fidèlement le comportement du système embarqué.

Si tu veux, je peux aussi te faire :

-  version PDF prête à envoyer
-  version “rapport de stage industrie”
-  schéma propre style architecture diagram
-  ou une version beaucoup plus courte (1 page cheat sheet) Si tu veux, je peux aussi te faire :
-  version PDF prête à envoyer
-  version “rapport de stage industrie”
-  schéma propre style architecture diagram

✓ ou une version beaucoup plus courte (1 page cheat sheet) Si tu veux, je peux aussi te faire :

✓ version PDF prête à envoyer

✓ version “rapport de stage industrie”

✓ schéma propre style architecture diagram

✓ ou une version beaucoup plus courte (1 page cheat sheet)